

Hierarchical Coordination Multi-Agent Reinforcement Learning With Spatio-Temporal Abstraction

Tinghuai Ma , Kexing Peng , Huan Rong , Yurong Qian , and Najla Al-Nabhan 

Abstract—Many real-world cooperative problems can be implemented using Multi-Agent Reinforcement Learning (MARL) techniques, such as urban traffic control or multi-role games. However, the policy learning of MARL algorithms contains features of long-trajectory training and partial observability, which leads to the sparsity of reward and the lack of decision information. To solve the above issues, this article studies hierarchical deep MARL and proposes a novel model named Hierarchical Spatio-Temporal Communication Network (HSTCN). HSTCN designs hierarchical policies with two-time granularities: high-level and low-level policies. All agents are jointly entered into a joint policy containing the above two policies, and each has its execution policy. Specifically, the high-level policy provides intrinsic goals and continuous reward samples for the low-level policy to alleviate reward sparsity. The Low-level policy absorbs the above information to improve the efficiency of the agents' execution policies and interact with the environment to optimize the next reward. What's more, the high-level policy designs a graph-like structural model with Spatio-Temporal abstract. The Spatio-Temporal model expands receptive fields to receive neighborhood information and facilitates learning more robust policies by capturing the underlying graph's spatial dependencies and temporal dynamics. Meanwhile, an evaluation network is added to increase the robustness. Empirically, we demonstrated the effectiveness of HSTCN in a long-trajectory training environment through Simulation of Urban MObility (SUMO), while StarCraft II maps are tested as abstract environment. The above experimental results prove that the performance of HSTCN is superior to other advanced algorithms and verify the rationality of HSTCN design.

Index Terms—Multi-agent reinforcement learning, spatio-temporal, hierarchical reinforcement learning, communications.

Manuscript received 4 November 2022; revised 28 May 2023; accepted 22 July 2023. Date of publication 11 September 2023; date of current version 23 January 2024. This work was supported in part by the National Key Research and Development Program of China International Technology Cooperation Project under Grant 2021YFE014400, in part by the National Natural Science Foundation of China under Grants 62372243 and 42175194, and in part by Jiangsu Provincial Graduate Research and Practical Innovation Program under Grant KYCX23_1358. (Corresponding author: Kexing Peng.)

Tinghuai Ma is with the School of Software, Nanjing University of Information Science & Technology, Nanjing, Jiangsu 210-044, China (e-mail: thma@nuist.edu.cn).

Kexing Peng is with the School of Computer Science, Nanjing University of Information Science & Technology, Nanjing, Jiangsu 210-044, China (e-mail: 20211155004@nuist.edu.cn).

Huan Rong is with the School of Artificial Intelligence, Nanjing University of Information Science & Technology, Nanjing, Jiangsu 210-044, China (e-mail: ronghuan@nuist.edu.cn).

Yurong Qian is with the Xinjiang University, Urumqi 830-008, China (e-mail: qyr@xju.edu.cn).

Najla Al-Nabhan is with the Department of Computer Science, King Saud University, Riyadh 11362, Saudi Arabia (e-mail: nalnabhan@ksu.edu.sa).

Digital Object Identifier 10.1109/TETCI.2023.3309738

I. INTRODUCTION

MULTI-AGENT Reinforcement Learning (MARL) can be applied to real-world cooperative problems [1], including Artificial Intelligence (AI) for games [2] and traffic network optimization [3]. To solve interesting problems in the real world, agents must be good at planning and reasoning over long periods. For example, exploring a hybrid autonomous traffic control setting is delicate because each time step action does not immediately improve the traffic situation and its effect will not be determined until many time steps in the future. Due to the long-trajectory training, the reward cannot be evenly distributed to each time step, which leads to the sparse reward obtained by the agent and affects the decision. Hierarchical Reinforcement Learning (HRL) [4] based on time granularity tries to alleviate the problem of sparse reward. HRL allows learning goal-directed actions from sparse feedback in complex environments. HRL divides the entire task into independent goals by time sequence and trains a high-level policy to select goals, while a low-level policy learns how to reach each goal. In the learning process, the low-level policy will get the intrinsic reward of the high-level policy, which can significantly improve decision-making efficiency. In other words, the low-level policy can be rewarded continuously, alleviating the problem of reward sparsity. However, in traditional HRL work, the concurrent optimization of high-level and low-level policies easily leads to the instability of the training process and serious performance degradation.

Meanwhile, the information agents can observe is very limited in real-world cooperation tasks. For example, in the traffic signal control task, each intersection can only observe the traffic flow information at its intersection. In contrast, the change of signal lights adjacent or even in the entire traffic environment is mutually affected. The solution to the above problem is to develop a coordinated control policy to obtain the optimal cooperative action. A possible approach is to model the whole multi-agent system as a topological graph and treat agents as nodes to communicate with each other and reconstruct relationships [5]. The graph provides a rich and expressive approach to modeling and reasoning for multi-agent systems. Graph Neural networks (GNNs) [6] exploit the capabilities of existing deep learning techniques to learn from non-Euclidean graph-structured data. The advantage of GNNs can deal with graph structure directly and simultaneously carry out information propagation and parameter update

between the nodes and neighbors, which can alleviate the problem of insufficient information caused by partial observation values in a multi-agent environment. However, most existing studies only focus on spatial dependence and ignore the temporal dynamic effect of the whole graph [7]. The correlation between this spatial and temporal information is often overlooked. However, capturing such Spatio-Temporal dependencies can provide essential insights into the interactions between agents and the evolution of agents' actions over time, which helps agents consistently learn more robust coordination policies.

This article studies hierarchical deep MARL and proposes a model named Hierarchical Spatio-Temporal Communication Network (HSTCN). HSTCN developed a two-level policy: high-level and low-level policies, combined are called joint policy. The two policies work at different time granularities and communicate at fixed time steps. Specifically, The low-level policy is evaluated and trained by Centralized Training Decentralized Execution (CTDE) mode at the primitive fine-grained time, with the input of multi-agents in the form of union [8]. The significance of this design is agents need to be trained centrally to find low-level globally optimal cooperative policies while finding scalable decentralized systems that enable agents to choose low-level actions independently [9]. On a coarse-grained time, the high-level policy takes the agents' high-level decisions and the graph structure of the agent system as input, which encapsulates spatial and temporal dependencies. We design a Spatio-Temporal module to characterize the relationship between agent decisions in both spatial and temporal domains, aggregate the decision information of agents and generate high-level coordinated decision recommendations. At the same time, the high-level policy provides goals and intrinsic rewards for the low-level policy at a fixed time step, ensuring the reward is time intensive. In addition, adding an evaluation network with the global state value based on the high-level time granularity provide robustness for joint policy training. Thus alleviating the key limitation of the previous HRL work – the instability of training.

To verify the effect of our proposed method on coordinating traffic simulation environment, we use a high-frequency micro simulator, Simulation of Urban MObility (SUMO) [10]. SUMO has the essential characteristics of large-scale, long-trajectories, where an agent represents each traffic light (or intersection), the traffic state is regarded as the environment, and the traffic signal change is regarded as the action. This article simulates the traffic environment near the Nanjing Yangtze River Bridge. The average number of congestion at each intersection is used as the evaluation index of reward. Meanwhile, our method is applied to StarCraft II [11]. The characteristics of this environment include fewer time steps in each episode, dynamic surrounding information, etc. The results show that our method is superior to other excellent MARL algorithms. The model can optimize problems in many complex environments and has meaningful, practical application value.

The main contributions of this article are as follows:

- We propose a novel hierarchical structure combined with the CTDE mode to solve the sparse reward problem and improve the efficiency of sampling reward in the multi-agent cooperative task. An evaluation network of state value is

complemented with the joint policy training network to enhance robustness.

- We propose a Spatio-Temporal module in the high-level policy, which encapsulates spatial and temporal dependencies in a multi-agent system to extend the receptive field to enhance information communication, and capture relationships over time.
- The experimental results show that the HSTCN algorithm outperforms the advanced baseline MARL algorithm in learning efficiency and cooperation performance, both in the long-trajectory environment of simulated traffic and the abstract battle game environment.

This article is organized as follows. The Related Work and Preliminaries are presented in Section II and III. Overall architecture of HSTCN is presented in Section IV. Description of the Experimental Results are provided in Section V. Finally, we summarize our findings in Section VI.

II. RELATED WORK

A. Hierarchical Reinforcement Learning With Multi-Agent System

Hierarchical Reinforcement Learning (HRL) tries to solve the sparse reward problem in Single-Agent Reinforcement Learning (RL) [12]. When the agent is in the long-trajectory training, the environment reward feedback is sparse, which leads to learning difficulty. HRL divides the overall policy into high-level and low-level policies based on time granularity. At present, HRL can be divided into two categories: Option-based and Goal-based. Option-based represents a policy with temporal abstraction. Goal-based represents what the low-level policy needs to achieve. Specifically, the high-level policy provides options/goals and intrinsic rewards over a longer time span than the low-level policy. In comparison, the low-level policy selects actions in a shorter time span, thus alleviating the problem of reward sparsity to a certain extent. The specific operation is: the output space of the high-level policy is the goals' space. The low-level policy accepts the goals of the corresponding time period and outputs the primitive action. The high-level policy observes the environment state and generates a high-level action known as a goal in Data-Efficient Hierarchical Reinforcement Learning (HIRO) [13]. The goal of HIRO is defined within the state space. In addition, the simultaneous learning of the two levels of policies can easily lead to the instability of training. HAAR [14] used the Actor-Critic algorithm on policy to calculate auxiliary reward based on advantages to improve stability.

In recent years, HRL has been gradually applied to multi-agent systems. Feudal Multi-agent Hierarchies (FMH) [15] attempts to model the multi-agent environment as a feudal architecture. However, its main drawback is that it cannot be applied in the context of shared rewards. The present work adapts to the cooperative environment by combining the CTDE pattern of value-decomposition for credit allocation of all agents. Mahajan et al. [16] propose a new method called MAVEN, which mixes value and policy-based techniques by introducing a potential space of hierarchical control. The core idea of MAVEN is to let agents constrain their behavior based on potentially shared

variables controlled by the hierarchical policy. RODE [17] explicitly partitions the action space, with each action subspace corresponding to a kind of “role.” Aside from the high cost of adopting clustering, this is a novel idea. Inspired by task decomposition in the above work, HSTCN proposes a new framework of multi-agent cooperatives based on HRL.

B. Communication Methods in Multi-Agent Reinforcement Learning

In the cooperative problem of MARL, one work is to optimize the information interaction between agents, which includes explicit communication and information sharing. Traditional solutions for explicit communication include Differentiable Interactive Agent Learning (DIAL) [18] and Counterfactual Multi-Agent Policy Gradients (COMA) [19]. To understand the operation of other agents, DIAL and COMA set up a single network for each agent and transmit average messages through the agent network. Kim et al. [20] proposed an Intension Sharing (IS) scheme, in which each agent generates an imaginary trajectory by simulating other agents’ dynamic behaviors. Every agent captures its future action plan by applying the attention mechanism, and generates intentional information to communicate. Multi-Actor-Attention-Critic (MAAC) [21] proposes an Actor-Critic [22] based algorithm that trains decentralized policies (whose parameters are not shared) in a multi-agent environment. HAAR uses centrally computed Critics, and shares an attentional mechanism for selecting relevant information for each agent at each time step. These works are excellent but each agent maintains a complex network with different parameters, so they are not feasible for large-scale network environments. Another class of work uses global information sharing for average communication. Communication Neural Net (CommNet) [23] uses continuous communication for fully cooperative tasks, which works well despite its simplicity as a landmark work in inter-agent communication. In recent advanced work, Game Abstraction Network (G2ANet) [24] is a novel game abstraction mechanism based on a two-stage attention network, soft-attention and hard-attention, by modeling the agents’ relationship through a complete graph. HSTCN draws on the above work experience, designs a topological graph as a relational model for inter-agent communication, and uses an attention mechanism to assign observation credits.

C. Value-Decomposition Methods in Multi-Agent Reinforcement Learning

HSTCN can be summarized in an extension of MARL method, which is the scheme based on value-decomposition. In this scheme, the individual policy effect of each agent is evaluated by decomposing the global state-action value function, and the global reward is used to backpropagate the single-agent policy parameters. Traditional work Value-Decomposition Network (VDN) [25] tried to train a global state-action value network centrally, but a simple sum of individual policies leads to overfitting in complex environments. The subsequent work is optimized for this point. Rashid et al. [26] proposed

a framework with Centralized Learning and Distributed application Execution (CTDE) named QMIX. Through centralized information learning, the distributed policy of each agent is obtained. The global state information is used during training to improve the algorithm’s effectiveness. The QMIX designs a neural network to integrate the individual value function of each agent to obtain the global action-value function. QTRAN [27] by using proxy utility and global action value between the linear constraint ensures optimal decentralization, and adds a state-value network for decomposition aid. Most recent advanced works are optimized and adjusted based on the constraint method described above. The following works are optimized and redesigned based on the above framework to solve various MARL problems. GraphMix [28] is most relevant to our work, which combines the global state-action value function with a graph neural network module, which allows a single agent to be trained from the global reward. Wang et al. [29] proposed a Role-Oriented MARL framework (ROMA). In this framework, roles are set in real-time, and agents with similar roles tend to share their learning and deal with subtasks. Wang et al. [30] introduce the idea of value function decomposition into the framework of multi-agent behavior criticism (DOP). Based on this idea, DOP supports effective non-policy learning and solves the problems of centralized decentralized mismatch and credit allocation in discrete and continuous action space. LIIR [31] learns an intrinsic reward function for each agent, simulating the agent differently at each time step. HSTCN adopts the constraint structure based on QMIX to carry out the framework modeling of a two-layer policy, and designs the value evaluation network of QTRAN in reference.

III. PRELIMINARIES

A. Multi-Agent Reinforcement Learning in Dec-POMDP

The environment considered in this article can be described as a Decentralized Partially-Observable Markov Decision Process (DEC-POMDP) [32], and can be represented by a tuple $\langle S, A, P, r, O, N, \gamma, T \rangle$. In the Dec-POMDP environment, each agent can only get the local observation, but can not get the global observation of the global state. The global policy of agents determines the performance of the group, which is also common in practical tasks. To be specific, the environment state $s \in S$ is initialized at the start. At each time step t , based on the partial observation $o^i \in O$, each agent $i \in N$ selects the corresponding action $a^i \in A^i$. The preceding operations are determined by $O(s, a) : S \times A \rightarrow O$. This series of operations is determined by the state transition function $P(s' | s, a) : S \times A \times S \rightarrow [0, 1]$, where global action of all agents is defined as $a \in A \equiv A^i$. After the transfer operation is complete, the reward value is obtained $r(s, a) : S \times A \rightarrow R$. The multi-agent system aims to maximize the total expected return $\sum_{t=0}^{\infty} \gamma^t r_t$, where γ is the discount factor. Our approach is based on the value-decomposition framework, and low levels follow the assumption of Individual-Global-Max (IGM) [33], which assumes that the optimality of each agent’s individual value q function $\arg \max_{a^i} q^i(\tau^i, a^i)$ is consistent with the optimality of the global value Q function $\arg \max_a Q_{\text{tot}}(\tau, a)$.

TABLE I
DESCRIPTION OF NOTATIONS, MOST OF THE BOLD SYMBOLS REPRESENT THEIR JOINT FORMS

Subscript/ Superscript	Meaning
t, l	The low-level fine-grained primitive time step and the low-level run subscript.
T, h	The high-level coarse-grained time step and the high-level run subscript.
c	The number of layers convolved.
i, v	The current agent and its neighborhood agents.
Symbols	Meaning
o_t^i, o_T^i	The observation values obtained by each agent at low-level and high-level.
s_t^i, s_T^i	The state values of each agent with low-level and high-level.
τ_t^l, τ_T^h	The action-observation histories with low-level and high-level.
$a_t^{i,l}, a_T^{i,h}$	The action values of each agent with low-level and high-level.
r_t, r_T	The reward values of with low-level and high-level.
$q_t^l, q_{i,T}^h$	The individual action-observation values with low-level and high-level.
g_t^i, g_T^i	The goal of each agent with low-level form and high-level form.
Q^l, Q^h	The mixing nets with low-level and high-level.
B^l, B^h	The Replay Buffers with low-level and high-level.
π^l, π^h	The joint policies for choose actions with low-level and high-level.
r^l	The intrinsic reward generated by the advantage function.
V^h	The state evaluation network with high-level.
A^h	The advantage network with high-level.
$h_T^i, \hat{h}_T^i$	Agent i 's spatial relationships features and its further temporal relationships with neighbor agents.

The equation that describes IGM is as follows:

$$\arg \max_{\mathbf{a}} Q_{\text{tot}}(\boldsymbol{\tau}, \mathbf{a}) = \arg \max_{a^i} q^i(\tau^i, a^i), \quad \forall a \in A \quad (1)$$

where $\boldsymbol{\tau} \in \mathcal{T}$ represents the global action-observation histories of all agents, Q_{tot} is the global action-value function.

B. The Processing of Hierarchical Reinforcement Learning

Hierarchical Reinforcement Learning (HRL) [34] is a framework for alleviating the problem of sparse reward by interacting with decisions at different time granularities. The joint policy π^{joint} is composed of the high-level policy π^h and low-level policy π^l . π^h and π^l typically operate at different time granularities with coarse-grained T and fine-grained t . In a more direct form, π^h runs each T step to determine the goal in the next T step. Information needs to be communicated between the two policies, where the high-level π^h policy requires environment information as well as output from low-level π^l policy periodically, and π^l executes and trains with the intrinsic rewards and goals of high-level procedures.

With multi-agent temporal abstraction, we introduce hierarchical MARL as illustrated in Fig. 1. The high-level process can

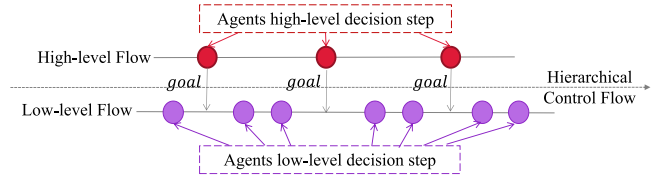


Fig. 1. Multi-agent environment description with two level's decision. The red and purple circles represent different periodic decision steps in the two levels.

be modeled as a Semi-Markov game [35], similar to the Multi-agent Semi-MDP (MSMDP). Formally, each agent i receives observation o_T^i and chooses a goal $g_T^i \in \mathcal{G}^i$ according to policy π^h , where $\mathcal{G}^i$ denotes the set of all possible intrinsic goals. A new goal g_T^i is selected until the current goal g_T^i is achieved. The next state s_T is determined by the global high-level action a^h according to π^h . The objective of π^h is to maximize the cumulative high-level reward r_T .

In contrast, the low-level policy π^l is set in the same way as the DEC-POMDP, except that it receives the intrinsic reward and goals of the high-level policy.

C. Spatio-Temporal Abstraction

In the setting of Spatio-Temporal modeling, we take Graph Convolutional Network (GCN) as the spatial abstraction for the multi-agent system, and temporal block combined with the hierarchical framework as the temporal abstraction.

The graph effectively represents relationships such as physical systems, social networks, road networks, protein interaction networks, etc. We denote a graph as $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of vertices, and $\mathcal{E}$ is a set of edges. Let $e_{iv} = (i, v)$ denote an edge connecting vertex i and vertex v . The set of neighbors of vertex v is denoted as $\mathcal{N}(i) = \{v \mid (i, v) \in \mathcal{E}\}$.

A GCN can aggregate neighborhood information based on the graph structure and leverage convolution operators. As multiple graph convolutional layers are stacked, neighbors within multiple skips can be involved in information aggregation. A GCN takes the original features of each vertex and the graph structure as input, and can output features for each vertex, each edge, or the overall graph according to the need of a task, such as node classification, link prediction, or graph classification, respectively.

GraphSAGE was proposed as a general aggregation-based graph convolution model [36]. In the c -th convolutional layer, features are aggregated as:

$$\begin{aligned} h_{\mathcal{N}_i}^c &= \text{AGG}_c(\{h_v^{c-1}, \forall v \in \mathcal{N}(i)\}) h_i^c \\ &= \varphi(\text{linear}^c([h_i^{c-1}, h_{\mathcal{N}_i}^c])) \end{aligned} \quad (2)$$

Where AGG_c denotes an aggregation function, e.g., mean or max-pooling function; h_i^c denotes the features of vertex i output by the c -th hidden layer; $[\cdot]$ denotes concatenated vectors; linear^c denotes a linear function, and φ denotes a nonlinear activation function. Specifically, in each convolutional layer, GraphSAGE first aggregates features of the neighbors for each vertex using an aggregation function. Then, it concatenates the

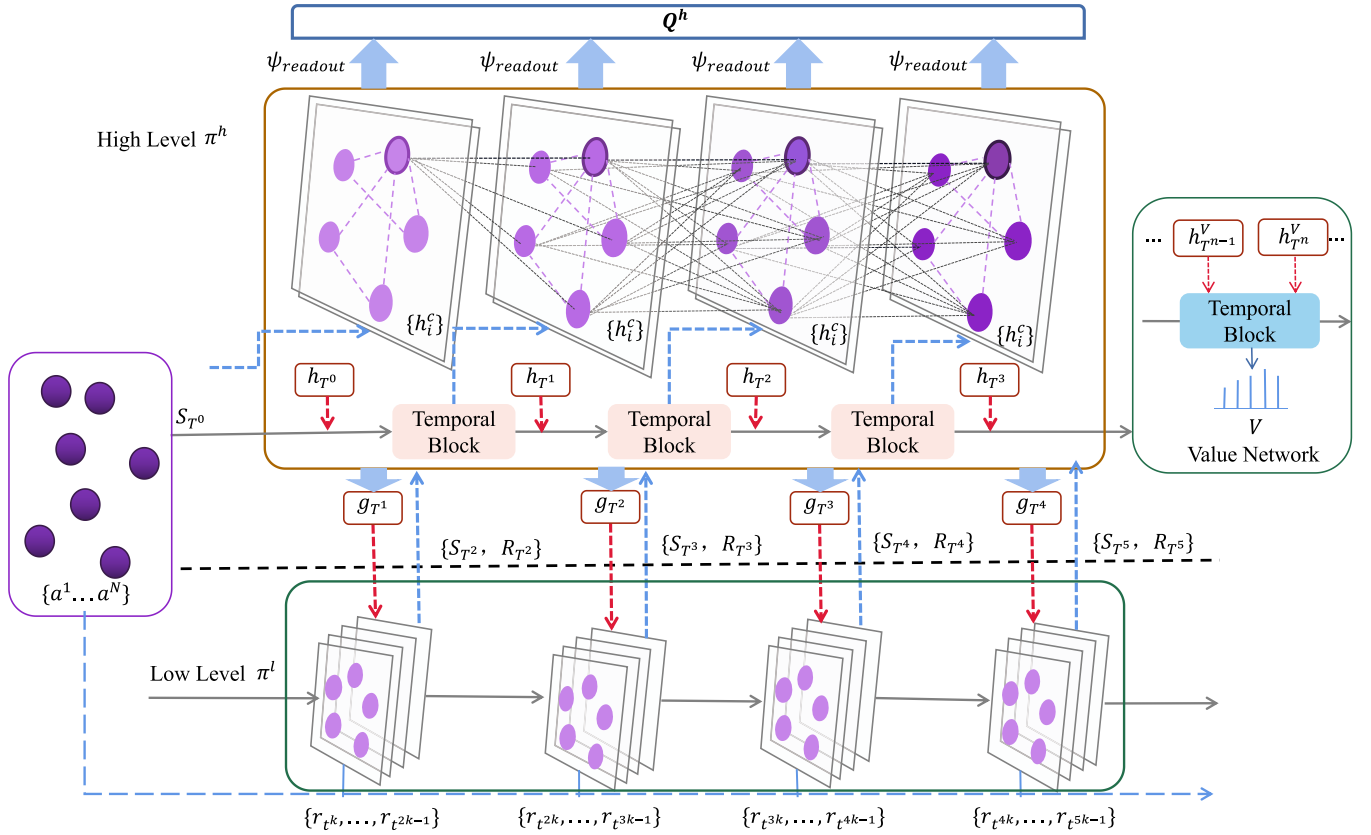


Fig. 2. Framework of HSTCN. The blue dashed line represents the input, the red dashed line represents the output, and the gray solid line represents the progress over time.

aggregated neighbors' features with the vertex's features, and uses the concatenated features to compute the output features of this vertex.

Sparse and delayed feedback is a significant challenge in learning effective policies. To tackle this problem, we introduce temporal abstraction, which is the key to solving long-period tasks since it reduces the difficulties by allowing agents to ignore irrelevant details. Taking a temporal view of the reinforcement learning process is common in HRL, which preserves the multi-agent properties and reduces the difficulty of decision-making. Thus agents can learn to cooperate and coordinate with others at a high level. In addition, reinforcement learning is a sequential decision, so modeling time in a policy to reference information from previous steps is helpful for action generation. Based on the popular Spatio-Temporal modeling, a temporal block: Long Short-Term Memory (LSTM) module [37] is added. The temporal block can be replaced with another sequential model, such as GRU or GRU-cell [38].

D. Scaled Dot-Product Attention Mechanism

The essence of the attention mechanism can be described as the mapping of a query to a series of key-value pairs. Among various attention mechanisms, scale-dot attention (i.e., query $\tilde{Q}$, key K , and value V attention) is one of the well-known

self-attention mechanisms [39].

$$\text{Attention}(\tilde{Q}, K, V) = \text{softmax}\left(\frac{\tilde{Q}K^T}{\sqrt{d_k}}\right)V \quad (3)$$

IV. PROPOSED SCHEME

This section describes the whole operation flow of HSTCN. The high-level policy π^h runs with the coarse-grained T , training every k steps and feeding information to the low-level policy. After π^h selects the global high-level action a^h and goal g_T , π^l will select the global primitive action a^l depending on g_t for k steps and the observation o in real-time t . In Dec-POMDPs, all agents share a reward value given by environments, and we denote it as the original reward r_t of π^{joint} . We also set the high-level reward function as $r_T = \sum_{t=0}^{b-1} r_{T,k+b}^t$. We denote the replay buffers of both level policies as B^l and B^h respectively, and the stored trajectories correspond to $\langle s_t, o_t, o_{t'}, g_t, a_t^l, r_t \rangle$ and $\langle s_T, o_T, a_T^h, r_T, o_{T'} \rangle$. The overall architecture is shown in Fig. 2, in which the dashed black line in the middle of the figure represents the dividing line between high-level and low-level execution. The solid box above the boundary line demonstrates the high-level policy model in the order of coarse-grained time steps. The high-level policy encapsulates the Spatio-Temporal module with GCN as the spatial abstraction and Temporal Block as the temporal abstraction. The solid box below the boundary

is the training and execution of the low-level policy model in fine-grained time steps. Their input is the information of the current agents, and all agents enter the high and low policies model in the form of joint input, then transmit and execute their individual policies, respectively. In other words, each agent has high and low policies π_i^h, π_i^l . Table I describes all the notations used in our method to aid readers' comprehension.

A. Low-Level Architectures

In a cooperative environment with a homogeneous agent system, the low-level policy is founded upon the CTDE pattern, with decentralized agent execution. In this setting, the offline policy of each agent operates independently while sharing parameters. However, in a heterogeneous agent system, the parameters are not shared. Its principal function is to output actions and interact with the environment. In our experiments, we implement the architecture with QMIX [26]. A low-level network takes the output of the individual action-value function $\{q_i^l\}_{i=0}^N$ as inputs and mixes them monotonically, producing the low-level global value Q^l :

$$Q^l(o_t, a_t, g_t) = \mathcal{M}(q_0^l(o_t^1, g_t^1, a_t^1), \dots, q_N^l(o_t^N, g_t^N, a_t^N)),$$

$$\frac{\partial Q^l}{\partial q_i^l} \geq 0, \forall i \in N \quad (4)$$

where o_t, a_t, g_t are the global observation, actions and goals of agents. This ensures that a global argmax operation performed on Q^l yields the same result as a set of individual argmax performed on q_i^l .

It minimizes the loss with a small batch of samples in the replay buffer B^l , with low-level policy π^l with it parameter θ

$$\mathcal{L}(\theta) = E_{(o_t, a_t, g_t, r_t) \sim B^l} [(y_t - Q^l(o_t, a_t, g_t; \theta))^2] \quad (5)$$

where $y_t = \gamma r^l + \gamma \max_{a_t} Q^l(o_t, a_t, g_t; \theta^-)$. Here θ^- denotes the parameters of the target value function and superscripts of agents are omitted for clarity. The Low-level architecture is compatible with both synchronous and asynchronous termination models.

B. High-Level Architectures

In the high-level policy, we organize the multi-agent system as an un-directed graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes and $\mathcal{E}$ is the set of edges. Each node $i \in \mathcal{V}$ represents either an agent i or an entity. Entities represent objects in the environment such as landmarks, obstacles, etc. In practice, a given agent's behavior is more likely influenced by the agents / entities in its immediate vicinity. Thus, the communication range of an agent can be specified as a function of proximity. The setting for modeling the multi-agent system as a graph is consistent with the Section III Preliminaries. Further, for a given agent i , its neighborhood is denoted by $\mathcal{N}_e(i)$ respectively.

To make it easier to understand, a single agent is used as an example for analysis. At any given time step T , the local observation o_T^i perceived by agent i including position and velocity.

The raw input o_T^i is encoded into a fixed-length state vector $x_T^i = \phi_O(o_T^i)$, where $\phi_O(\cdot)$ denotes the encoding function.

For each neighbor $v \in \mathcal{N}_e(i)$, agent i computes the encoding $e_T^{iv} = \phi_E(o_T^{iv})$, where $\phi_E(\cdot)$ denotes the neighbor entity encoding function. Here, o_T^{iv} represents i 's observation of N (i.e. the relative position of neighbor v with respect to agent i). As o_T^{iv} is determined by considering a given agent's own perception of its neighbouring entities, explicit communication between the agent and neighbours is not required. Each agent i , linearly transforms the encoding x_T^i into $\tilde{Q}_i$ and every encoding e_T^{iv} into K_{iv} and V_{iv} . Similar to a graph attention mechanism is utilized to compute attention weights $\alpha_{iv} = \text{softmax}(\frac{\tilde{Q}_i K_{iv}^\top}{\sqrt{d_K}})$, d_K is the dimensionality of K . A fixed-length output vector $\tilde{e}_T^i$ is computed as a weighted sum of the V_{iv} values with the attention weights. The node feature vector h_T^i associated with each agent i at time step T , is obtained by concatenating $q_{i,T}^h$ with $\tilde{e}_T^i$.

$$h_T^i = [q_{i,T}^h \| \tilde{e}_T^i] \quad (6)$$

where $\|$ denotes the concatenating operation. Intuitively, h_T^i represents agent i 's perception of its own local state as well as the agent within its neighbourhood.

At each time step T , an internal propagation process is repeated several times to capture the spatial correlations among agents coherently. Thus, the node feature vector h_T^i is considered as the node feature vector of agent i at propagation step 0 (in time step T) and is re-written as $h_{T,0}^i$. Note that, in deriving the propagation module, we drop time T for notational simplicity. Thus, $h_{T,0}^i \rightarrow h_0^i$.

Let $\hat{h}_T^i$ denotes the final node feature vector of agent i obtained after a fixed number of propagation steps (at time step T). This section uses the Temporal Block combined with historical information to try to capture the dynamic characteristics of time. In particular, we use LSTM (denoted by $\phi_T(\cdot)$) for this purpose.

Intuitively, the output $\hat{h}_T^i$ represents an enriched representation of the raw observation o_T^i perceived by a given agent i . That is, $\hat{h}_T^i$ is more reflective of the spatial influences of the agent-agent interactions as well as the underlying temporal dynamics of the system. As shown in Fig. 3, the embedding process described above analyzes the relationships and characteristics of nodes and neighbors, and then enters the convolution model, which can be seen in the high-level policy module in Fig. 2. The first layer convolution process can be written as:

$$h_i^c = \psi_{combie}^c \left(\sum_{v \in \mathcal{N}(i)_+} \alpha_{iv}^c \hat{h}_T^i W_i^c \right) \quad (7)$$

By overlaying c convolution layers, agent i can aggregate the information of nodes with a distance of $c - hops$, where $c \in \{1, \dots, C\}$. $\psi_{combie}^c(\cdot)$ represents a monotonically increasing (and possibly nonlinear) parameterized combinatorial function. h_i^c is the potential feature of agent i generated by the convolution layer, and W^c represents the trainable weight in the convolution layer, α_{iv}^c represents self-attention weights for $\hat{h}_T^i$. After several updates, the node knows more about the characteristics of itself and its neighbors. This also allows the creation of an accurate

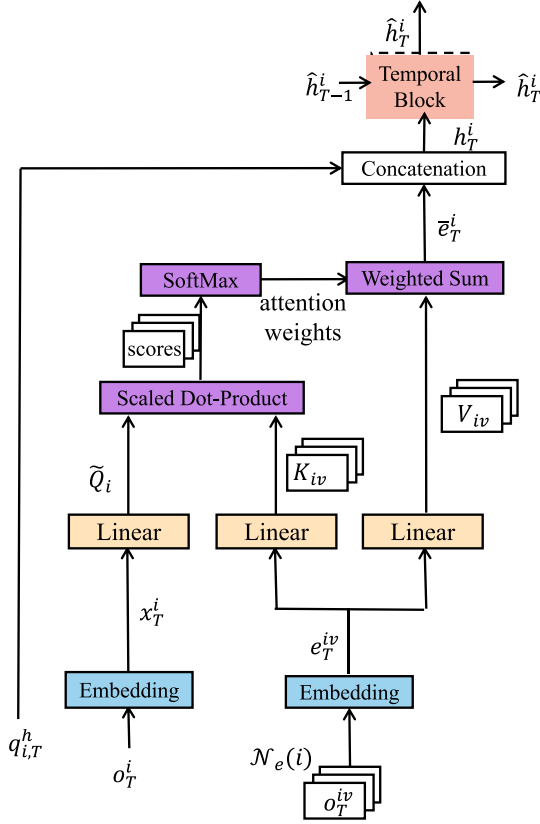


Fig. 3. Input encapsulation of Spatio-Temporal module.

representation of the entire graph. After two layers of convolution, the output is:

$$Q^h(\tau_T, \mathbf{a}_T) = \psi_{readout}(\{h_i^c\}_{n \in V}) \quad (8)$$

Where $\psi_{readout}$ maps the relationship of the whole graph and the heterogeneous characteristics of nodes to the global state-action value function, which is a graph readout operation (such as average/max pooling). It is worth noting that, in order to achieve the desired effect of mixed monotonicity, we ensured that all the activation functions used in all the attention mechanism are non-negative. Additionally, the weight matrices W_i^c in the graph convolution are also non-negative. These choices contribute to maintaining the desired properties of mixed monotonicity and further enhance the effectiveness of our model.

The above process represents a single propagation step (one-hop) in which a given agent i acquires information from its neighboring agents. Thus, repeating the process a fixed number of steps (a multi-hop propagation mechanism) enables a given agent to increase its receptive field gradually. Specifically, a sparsely connected multi-agent network enables an agent to propagate information to agents outside its communication range. After the global Q -value is computed, the g that needs to be fed to the low-level policy is determined by the individual q -functions. The goal can be intuitively understood as a recommendation for improving action selection at the low level.

C. Evaluation Network and Loss Functions

To define the intrinsic rewards that high-level policies give to low-level policies, we refer to the dominance function Settings of HAAR [14] and QTRAN. The low-level joint policy is trained using the QMIX framework exclusively, while the advantage function of high-level is designed to primarily generate intrinsic rewards that assist in the training of the low-level. When the state is s_T , and π^h performs the global action $\mathbf{a}_T^h$, the advantage function is set as $A^h(s_T, \mathbf{a}_T^h)$. The advantage function runs on a coarse-grained time step T , defined as:

$$A^h(s_T^h, \mathbf{a}_T^h) = E_{s_{T'} \sim (\pi^h, \pi^l)} [r_T + \gamma V^h(s_{T'}) - V^h(s_T)] \quad (9)$$

where $V^h(\cdot)$ represents the evaluation network of state value, which gives the expected return if agents start in a certain state and always act according to the current policy π^{joint} . Therefore, this function requires an additional neural network for estimation. We calculate the individual state value $V_i^h(\tau_T^{h,i})$ of all agents and then feed them into the global state value net to finally get the function $V^h(s_T)$. The state-value function $V^h(s)$ judges only the optimal value of the current state value.

For π^l , the advantage function A^h is evenly divided among k steps to get the intrinsic reward of each step, which can be calculated by the following formula:

$$r^l = \frac{A^h(s_T, \mathbf{a}_T^h)}{k} + r_t, \quad T \cdot k \leq t < (T') \cdot k \quad (10)$$

Note that because of the individual network parameter-sharing mechanism between agents, the complexity of the two added sets of neural networks does not linearly increase with the number of agents. In addition, since the low-level policy runs at the primitive fine-grained time step, while the high-level policy is trained at the coarse-grained time step, the time complexity does not increase significantly compared to the flat structure.

At the low-level, the training is conducted on a step-by-step basis, resulting in continuous abstractions. In this continuous setting, the advantage function may tend to converge to a flat value, potentially diminishing its effectiveness. On the other hand, the high-level training occurs periodically, generating discrete abstractions of the environment. In this discrete setting, the advantage function serves as a guidance mechanism, allowing us to normalize potentially oscillating outputs and align them with more stable estimates. Additionally, the high-level provides enhanced rewards to the low-level, further obviating the need for using the advantage function at the low-level. By exclusively utilizing the advantage function at the high-level, we aim to leverage its benefits in terms of providing guidance, normalizing potentially oscillating outputs, and enhancing the rewards given by the high-level to the low-level. This approach is specifically tailored to the discrete abstractions and periodic training of the high-level, while effectively addressing the training dynamics and coordination requirements within our framework.

From the above explanation, we can get the loss function of the three sets of neural networks: the high-level state-value network, the high-level action-value network, and the low-level action-value network. ψ , ϕ , and θ represent their parameters,

Algorithm 1: Calculation Process of HSTCN.

```

1: Initialize high-level Mixing Net  $Q^h$ , low-level Mixing
   Net  $Q^l$ , high-level Agents Net  $q_i^h$ , low-level Agents Net
    $q_i^l$ , high-level replay buffer  $B^h$ , low-level replay buffer
    $B^l$  and  $V^h$ 
2: Initialize reward discount factor  $\gamma$ , target network update
   parameters  $\tau$ 
3: for episode = 1 do
4:   Random initialization action  $a_t$  to explore
5:   Get the global state  $s_t, \mathbf{o}_t$ 
6:   for  $t = 1, T$  do
7:     if  $t \bmod T = 0$  then
8:       if  $t > 1$  then
9:         Calculate  $r_T$ 
10:        Store  $\langle s_T, \mathbf{o}_T, \mathbf{a}_T^h, r_T^h, \mathbf{o}_{T'} \rangle$  in  $B^h$ 
11:      end if
12:      Select  $g_T^i$  by  $\epsilon$ -greedy  $(q_i^h(o_T^i, a_T^i)), \forall i \in N$ 
13:      if  $T \bmod T_{train} = 0$  then
14:        Update  $Q^h$  and  $q^h$  and  $V^h$  with  $B^h$ 
15:      end if
16:    end if
17:    Get  $a_t^i$  from  $\epsilon$ -greedy  $(q_i^l(o_t^i, g_t^i, a_{t-1}^i))$  for each agent
18:    Get  $a_t^l$  from  $s_t, \mathbf{o}_t, r_t$ 
19:    Calculate  $r_t^l$  for each agent
20:    For all agents, store  $\langle s_t, \mathbf{o}_t, \mathbf{g}_t, \mathbf{a}_t^l, r_t, \mathbf{o}_{t'} \rangle$  into
       low-level replay buffer  $B^l$ 
21:    if  $t \bmod t_{train} = 0$  then
22:      Update  $Q^l$  and  $q^l$  with  $B^l$ 
23:    end if
24:  end for
25: end for

```

respectively. Thus, the following loss function is obtained:

$$\mathcal{L}_V^h(\psi) = \left(r_T + \gamma_h \max_{\mathbf{a}_{T'}^h} Q^h(\tau_{T'}^h, \mathbf{a}_{T'}^h | \phi) - V^h(s_T | \psi) \right)^2 \quad (11)$$

$$\mathcal{L}_Q^h(\phi) = \left(r_T + \gamma_h \max_{\mathbf{a}_{T'}^h} Q^h(\tau_{T'}^h, \mathbf{a}_{T'}^h | \phi^-) - Q^h(\tau_T^h, \mathbf{a}_T^h | \phi) \right)^2 \quad (12)$$

$$\mathcal{L}_Q^l(\theta) = \left(r^l + \gamma_l \max_{\mathbf{a}_{t'}^l} Q^l(\tau_{t'}^l, \mathbf{g}_{t'}^l, \mathbf{a}_{t'}^l | \theta^-) - Q^l(\tau_t^l, \mathbf{g}_t^l, \mathbf{a}_t^l | \theta) \right)^2 \quad (13)$$

where ϕ^- and θ^- refer to the parameters of the high-level and low-level action-value target network, respectively. It is worth mentioning that the optimization of the three networks is independent of each other. The calculation process of HSTCN is as Algorithm 1 shows.

V. EXPERIMENTS

A. The Comparison Algorithms

In comparison algorithms, we choose well-known algorithms similar to the HSTCN method, introduced in the Related Work. Table II is a brief introduction to these methods.

B. Traffic System Control Experiments

SUMO: Simulation of Urban MObility, referred to as “SUMO”[10]. It is a traffic simulation software that can customize the road structure, connection, traffic lights, priority, number and speed of vehicles, etc.—designed to handle large networks. It allows multimodal transport simulation, including pedestrians, and is equipped with extensive tools for scene creation. The environment introduces VISUM, Vissim, Shapefile, OSM, RoboCup, MATsim, OpenDRIVE, and XML-Descriptions, and the heuristic determines the missing values. The Simulation of Urban MObility (SUMO) environment can output the required information during the simulation. SUMO is open source and widely adopted.

We choose SUMO as our primary testing environment because it is characterized by long training trajectories (typically 6-12 minutes per episode) and partial observability (each intersection can only observe its own traffic). A multi-agent system can represent a traffic model containing multiple traffic intersections, and each agent represents an intersection. Modeling a large-scale traffic network is complex because the information that each signal controller can observe is limited. Besides, the environment is unpredictable, the dynamic change of traffic flow is a complex task and each intersection affects the other. Since the policy of each traffic signal is constantly changing, the whole environment is non-stationary, which has the features of a multi-agent cooperation environment.

In SUMO, the target is to reduce congestion in the current scenario and minimize the length of queues at intersections. Therefore, the following steps are repeated throughout the multi-agent system:

(1) HSTCN initializes a policy according to the current environment state s_t ; (2) Performs actions according to the current policy and enters the exploration stage. (Switch traffic lights at each intersection); (3) The emulator executes several time steps according to the current random policy and stores the corresponding environmental rewards; (4) HSTCN uses stored rewards to optimize policy training.

1) *Designing the Agent:* An agent representing an intersection includes multiple traffic signals, so adjacent phase conflicts will not occur.

2) *States:* The state is defined as $s_t^i = \{\text{wave}_t[\text{lanes}_{vi}]\}_{\text{lanes}_{vi} \in \text{Lanes}^i}$, where Lanes^i is the set of lanes v converging at a signalized intersection (agent) i . And $\text{wave}[\text{veh}]$ measures the total number of approaching vehicles along each incoming lane, within 50[m] to a signalized intersection.

In TSC road networks, each agent observes the number of vehicles car_t^i at intersection, the waiting time wait_t^i , the queue length queue_t^i for each lane at time t . Let denotes the $\text{o}_t^i =$

TABLE II
COMPARISON ALGORITHMS

Framework	Features	Details
HRL	RODE [17]	Uses the method of assigning roles to decompose action space.
	MAVEN [16]	Introduces an action reference noise network.
Value-Decomposition	QMIX [26]	Proposes the decomposition of monotone valued functions.
	QTRAN [27]	Proposes an method to assign credit.
	GraphMix [28]	Proposes a value-decomposition model based on graph network.
	ROMA [29]	Uses the method of assigning roles to decompose tasks.
	DOP [30]	Decompose the centralized Critic into the form of a weighted sum of separate Critic.
	LIIR [31]	Has designed own reward mechanism.
Average communication	G2ANet [24]	Uses GNNs for communication selection.
	CommNet [23]	Communicates using information sharing.
	Central-V	A full-time communication basic method we add.
Multiple networks	COMA [19]	Emphasize the implicit communication function among agents.
	MAAC [21]	Proposes to use an attention mechanism to learn implicit communication content.

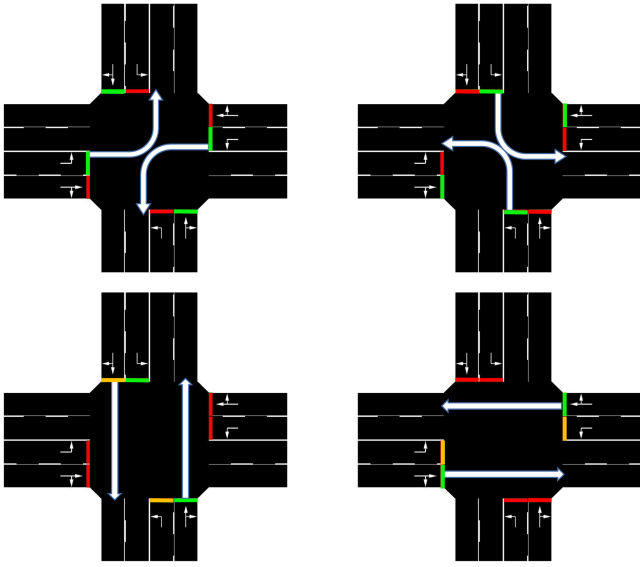


Fig. 4. Element composition of the simulated environment.

$\{Car_t^i, Wait_t^i, Queue_t^i\}$. We collect these characterizes in Δt :

$$Car_t^i = \sum_{t=0}^{\Delta t} car_t^i \quad (14)$$

car_t^i represents the car number of $agent^i$ at time t . Similar to the Car_t^i , $Wait_t^i$, $Queue_t^i$ is calculated severally. In the TSC environment, each agent considers the local observation o_t^i and the information about others from s_t^i . Basically, local observation is $o_t^i\{Car_t^i, Wait_t^i, Queue_t^i\}$. The whole TSC environment state formulates as $S_t = \{s_t^1, s_t^2, \dots, s_t^N\}$.

3) *Action Space*: Each agent can choose three different colors, green, yellow, or red at various intervals, and the behavior of the selected signal is treated as the action. HSTCN generates both the appropriate phase p of the traffic signals and its corresponding duration d^p .

The essential elements of the environment in the article are shown in Fig. 4. Each road at a four-way intersection is bidirectional and contains four lanes. At any time can turn right. Here, we only consider car traffic.

The action space comprises two hierarchical subspaces of traffic light phases and the associated phase durations. According to the greedy policy, each agent chooses the max action value a_t^i :

$$a_t^i = \operatorname{argmax} \left(a_t^{p1}, a_t^{p2}, a_t^{p3}, a_t^{p4} \right) \quad (15)$$

When the action is determined, the phase p_t^i will be compared with the previous phase p_{t-1}^i firstly, and if same as the previous one p_{t-1}^i , the phase will be executed continuously. If not, the traffic light will turn yellow and change to the next phase p_{next} , regarding the action.

$$p_t^i = \begin{cases} p_t^i, a_t^i = p_t^i \\ p_{next}, a_t^i \neq p_t^i. \end{cases} \quad (16)$$

4) *Reward Function*: Our reward value is called delay of vehicles, and the following factors are considered comprehensively:

- 1) Number of vehicles per intersection.
- 2) Average waiting time of vehicle waiting.
- 3) Rate of flow at the intersection.
- 4) Average speed of vehicles when exiting the intersection.
- 5) Average speed of vehicles when arriving at the intersection.

The reward function at time t cumulates the queues (number of vehicles with speed less than 0.1 m/s) at the lanes concurring to a certain signalized intersection computed at time $t + \Delta t$:

$$r_t = \sum_{ji \in N, l \in L_{ji}} queue_{t+\Delta t}[lanes] \quad (17)$$

5) *Training*: To make the training framework more complete, Pytorch is straightforward to set up and easily configured to work with a GPU. The training network parameters are shown in Table III. We have provided the average steps' rewards during the training process for each scenario, based on the average results from 15 independent experimental runs. The solid lines of Figs. 7 and 9 represent the median value of the learning curve, and Tables V and VI represent the rewards and variances after fitting.

Experiment 1: 2x2 intersections: The first test is conducted on a simple junction, shown in Fig. 5. The junction is composed of 4 intersections. Its purpose is to perform an initial performance

TABLE III
PARAMETER SETTING

Parameter	Value
Learning Rate	0.0001
Discount factor	0.9
Soft update factor	0.001
Optimizer	Adam
Memory length	50000
Sample size	64

TABLE IV
AVERAGE CUMULATIVE DELAY PER INTERSECTION AND AVERAGE QUEUE PER INTERSECTION ON 2X2 INTERSECTIONS

Controller	Average Cumulative Delay[s]	Average Queue [m]
Random	540	6
Central-V	380±23	4.8±1.2
COMA [19]	440±17	5.0±1.1
VDN [25]	330±26	4.6±1.3
MAAC [21]	410±31	4.8±1.5
GraphMix [28]	380±23	4.7±0.6
QMIX [26]	310±14	4.6±0.7
HSTCN	310±8	4.6±0.5

TABLE V
THE FITTING REWARD VALUE OF THE ALGORITHMS IN 4X4 ENVIRONMENT

Controller	Reward
Central-V	-310±21
CommNet [23]	-322±8
QTRAN [27]	-430±37
VDN [25]	-312±18
QMIX [26]	-285±9
G2ANet [24]	-288±14
GraphMix [28]	-291±11
HSTCN	-240±7

algorithm to observe their effects. During the evaluation, an average of 65 vehicles were inserted in 12 minutes (720 steps per episode), 1500 episodes in total.

Table VI shows the network's Average Cumulative Delay and Average Queue length of the environment with 2x2 intersections. Intuitively, the smaller both values are, the better the method works. The BOLD data is the best all-around performance, including large fitting values and minor shocks. The random method represents the performance of exploring without training using the reinforcement learning framework. In this simple simulation environment of four intersections, the fitting speed of all methods is almost the same. The final presentation effect will not have too much difference, improving efficiency compared with the random case. It can be seen that the methods based on value-decomposition (VDN [25], QMIX [26], GraphMix [28]) all perform well, and the policy of complete communication (Central-V) also performs well. However, the multi-network

TABLE VI
THE FITTING REWARD VALUE OF EACH INTERSECTION IN NANJING YANGTZE RIVER BRIDGE

Controller	Reward
Central-V	-1.10±0.03
COMA [19]	-1.22±0.24
QTRAN [27]	-1.18±0.29
MAVEN [16]	-1.23±0.16
QMIX [26]	-1.12±0.12
G2ANet [24]	-1.13±0.09
GraphMix [28]	-1.21±0.08
HSTCN	-0.64±0.06

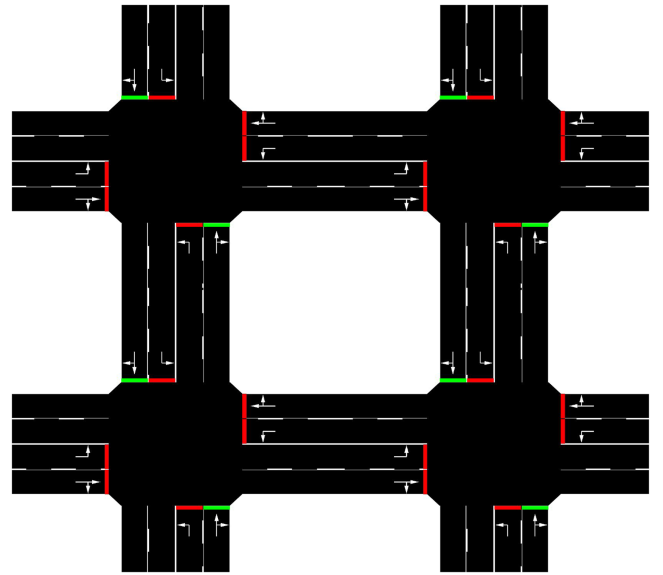


Fig. 5. Simulated environment of 2x2 intersections.

methods (MAAC [21], COMA [19]) is weak in the homogeneous environment, especially in the quasi-true environment with complex influencing factors.

Experiment 2: 4x4 intersections: The second test is shown in Fig. 6, which with 16 intersections, displays a much higher complexity than the intersection presented in the previous section. During the evaluation, an average of 410 vehicles were inserted in 12 minutes (720 steps per episode), 1500 episodes in total.

Fig. 7 shows the average reward curves for all methods. Random method represents the performance of exploring without training using the reinforcement learning framework. As the number of controllable agents and traffic increases, the performance of the different methods starts to change, the curve of QTRAN [27] is very irritated, which is analyzed in this article because the traffic situation is complex, and the multiple constraints of the algorithm appear the fitting problem. The reason for this analysis is that VDN [25] and QMIX [26] with relatively simple constraint Q function perform well. The fitting speed of the method based on value-decomposition (VDN, QMIX, GraphMix [28]) is generally higher than that of the method based on large-scale communication (G2ANet [24],

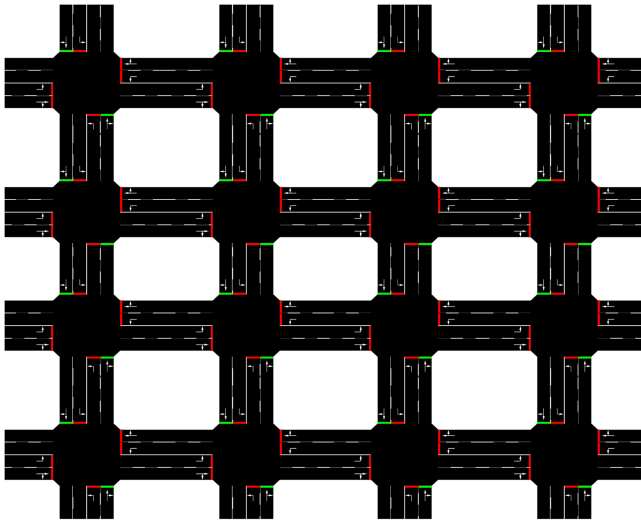


Fig. 6. Simulated environment of 4x4 intersections.

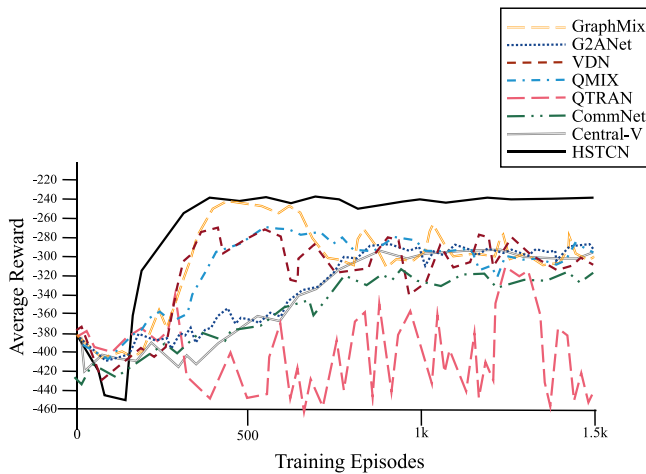
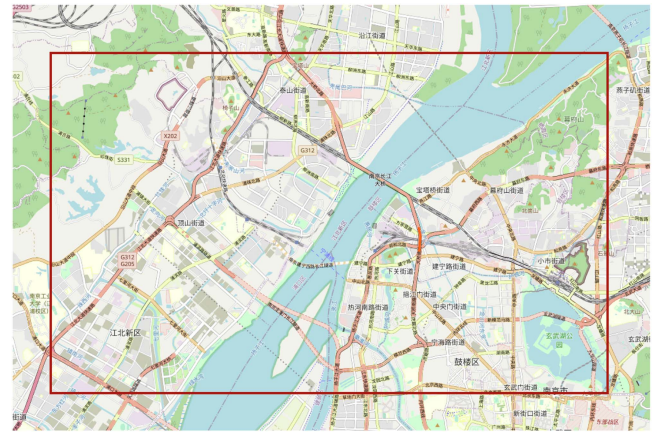


Fig. 7. Training curves of all methods in 4x4 intersections.

Central-V, CommNet [23]). We analyze this because large-scale communication is slower in extracting complex relations. As expected, HSTCN continues to perform very well. The results show that the comprehensive method helps improve the training effect. At the same time, among other algorithms, excluding HSTCN, GraphMix [28] performs well because of its graph structure analysis ability, which can model the problem well.

Experiment 3: Simulation of Nanjing Yangtze River Bridge: In Experiment 3, we selected the traffic area surrounding the Nanjing Yangtze River Bridge from OpenStreetMap and transformed it into a SUMO map in our system using a tool called NETCONVERT. The area displayed in the maps generated from OpenStreetMap, SUMO and NetEdit (the direct form of the road network) is shown in Fig. 8(a), (b), and (c). Taking the road network in some areas of Nanjing Yangtze River Bridge as the prototype, a TSC simulation system based on SUMO is designed. The general setting of the environment is as follows: Traffic light junctions have 41 intersections (agents can be



(a) OpenStreetMap expression



(b) SUMO expression



(c) NetEdit expression

Fig. 8. Simulated environment of Nanjing Yangtze River Bridge.

controlled), and an average of 830 vehicles were inserted in 6 minutes (360 steps per episode), 2000 episodes in total.

The reward curves during training in this simulation is shown in Fig. 9(a). In this scenario, the performance of COMA [19] is not outstanding, which may be due to a large number of traffic lights and complex observations, making it difficult for the multi-network in the method to backpropagate. In addition, the training curve of QTRAN [27] still fluctuate considerably. As

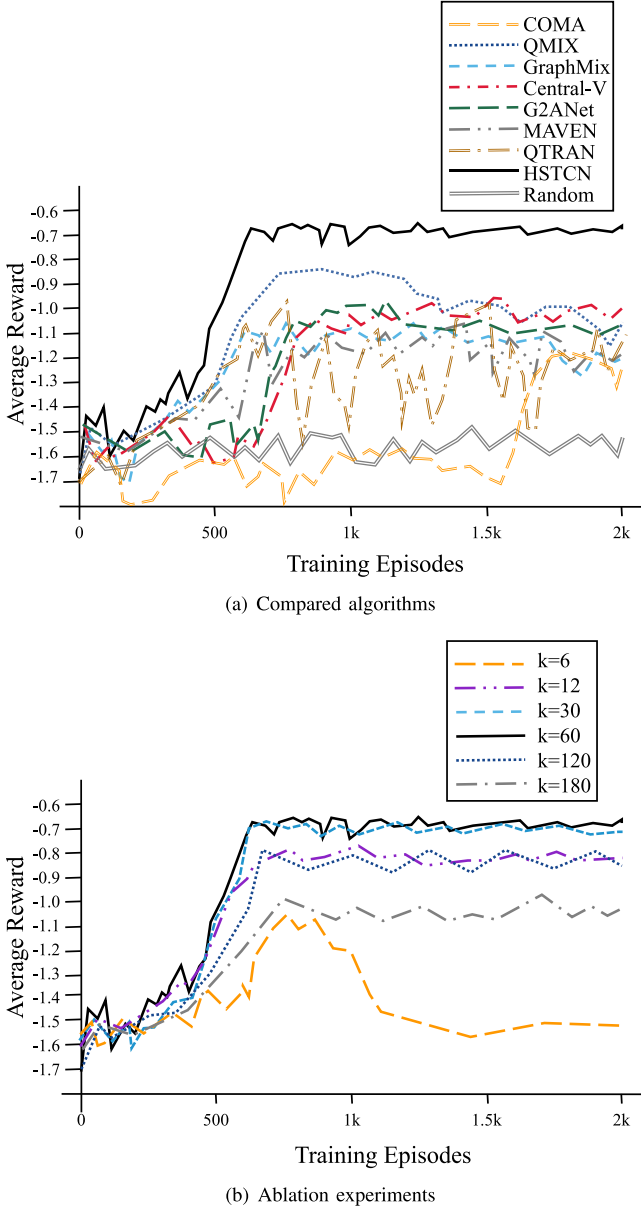


Fig. 9. Learning curves in the Nanjing Yangtze River Bridge environment.

expected, Our method achieves the best training performance. From the convergence rate, it can be seen that HSTCN can quickly adapt to the complex environment of large-scale agents. Our Spatio-Temporal network will respond soon when facing multiple agents and connect the related neighbor agents. Our method can quickly achieve smooth convergence in the training process, which benefits from the graph network, capturing the influence relationship of the updating agent in time. From the results of the three experiments, it can be seen that with the increase in the number of agents, HSTCN has more obvious advantages. This article analyzes that the reason for this phenomenon is that with the increase of agents' scale, the space of exploration and action increases. Because HSTCN has a hierarchical structure that divides time tasks, high-level policies can effectively divide the whole decision space into several smaller decision spaces, and because the Spatio-Temporal model

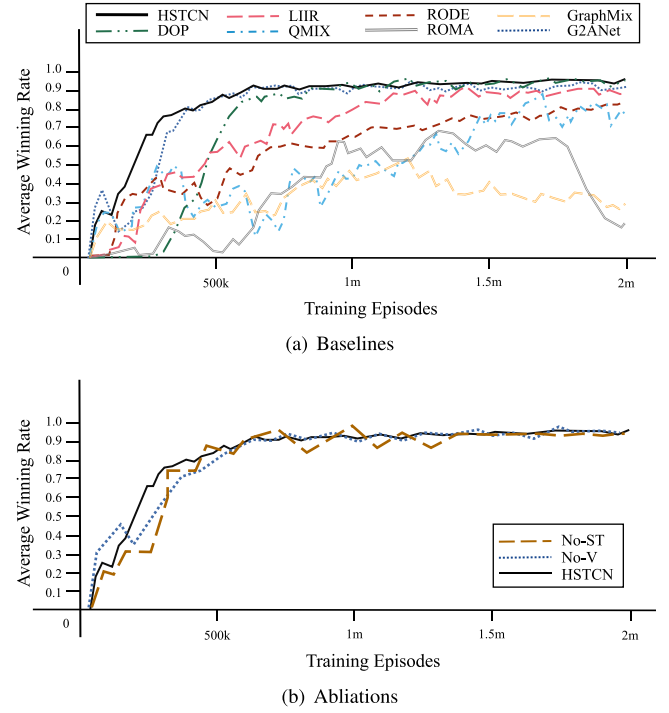


Fig. 10. Learning curves in StarCraft II of map 2s3z.

divides the neighborhood of agents, the information that agents need to consider is clearer. As a result, low-level policies can operate in a relatively small decision space with explicit domain knowledge.

Fig. 9(b) compares the performance of different k values of HSTCN under the current environment. k determines how often the low-level action space varies, and we discuss how k affects performance. It can be seen that the larger the training interval of the high-level policy, the better the effect will be, and it will finally converge to an optimal value. However, if k continues to increase on this basis, the performance of HSTCN will worsen. In other words, when k is too small, the high-level policy participates in the training execution of the low-level policy too frequently, which leads to the sub-optimal policy. When k is too large, the high-level policy almost outputs no intrinsic training information, resulting in the low-level policy that relies on its own training results to fit, which is close to the effect of the method without the hierarchical framework. The ablation experiment proves that the model training method based on time stratification can make the effect more robust. (The selection of k value is adjusted according to the number of training steps in the environment.)

In this simulation environment, we mainly test the adaptability of our method to a large-scale agent environment and the utilization efficiency of observable data in complex parts. We directly use this complex pseudo-real environment. HSTCN makes learning complex agent relationships easier and can achieve the best effect. In the research field of MARL, people are keen to generalize the algorithm in real problems, which is the ultimate goal of model research. In the practical application scenario of TSC, the exploration process of the MARL algorithm will cause security problems and material waste in actual society.

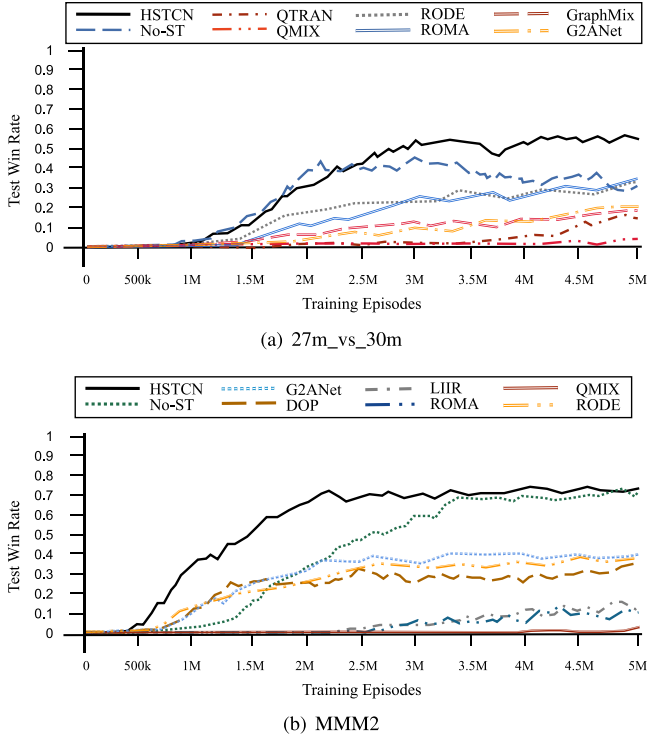


Fig. 11. Learning curves in StarCraft II of map MMM2 and 27m_vs_30 m.

Our model training and the fitting stage are carried out in the simulation to get close to true humanity and avoid resource waste. Although there is a particular gap between the simulator and the natural environment, we try our best to achieve the perfection of simulation and effect to facilitate the repeated testing of reinforcement learning methods.

C. StarCraft II

We evaluated the HSTCN in StarCraft II (SMAC) in a multi-agent challenge environment, which provides a different set of micromanagement challenges for the distributed multi-agent reinforcement learning method benchmark, “2s3z”. In this map, the agent teams of allies are controlled by MARL policy, while the built-in AI controls the enemy units in the game. In this regard, we have provided the average episode win rate during the training process for each scenario, based on the average results from 10 independent experimental runs. The solid lines of Figs. 10 and 11 represent the median value of the learning curves, and Tables VII and VIII represent the rewards and variances after fitting.

In addition, we tested it on the “27m_vs_30 m” and “MMM2” maps, both of which represent “super hard” difficulty levels, presenting different multi-agent cooperative challenges. In “27m_vs_30 m”, our team adjusted its strategy to effectively target damaged units and win with only 27 Marines against 30 homogeneous and asymmetric enemy Marines with fewer units. MMM2, on the other hand, features a heterogeneous and asymmetrical team of one medic, two Marauders, and seven Marines against an enemy team of one medic, three Marauders, and eight Marines, which allowed us to test the adaptability of

TABLE VII
WINNING RATE AND AVERAGE REWARDS IN STARCRAFT II OF MAP 2S3Z

Controller	Winning Rate	Rewards
DOP [30]	0.95±0.03	19.79±0.04
LIIR [31]	0.86±0.02	19.43±0.01
ROMA [29]	0.33±0.29	14.23±0.03
RODE [17]	0.81±0.01	19.02±0.02
QMIX [26]	0.72±0.01	18.94±0.03
G2ANet [24]	0.91±0.02	19.63±0.04
GraphMix [28]	0.35±0.13	15.64±0.52
HSTCN	0.96±0.01	19.81±0.03

HSTCN to a heterogeneous environment. In these two scenarios, we take the best results of three independent experiments of all the algorithms. We pause training every 5000 steps and run 32 evaluation sets using decentralized greedy action selection. The parameter setting is consistent with SUMO.

Fig. 10(a) shows the success rate obtained by HSTCN and the comparison algorithms during training. The curve shows the average value of the oscillation, and a total of 2 million time steps have been trained. Among them, the performance of G2ANet [24] combined with communication and GNNs network analysis is very similar to that of HSTCN, and GraphMix [28] is a bit poor because it does not contain communication content. Other methods based on value-decomposition also have good performance, but the fitting rate is slightly lower than HSTCN. In the dynamic environment, agents need to have a faster response speed, and communication is a good solution. It can be seen that the performance of ROMA [29] is a little worse. This article analyzes that it takes time to deploy the role grouping function. ROMA may require a communication module. The fitting rate of QMIX [26] is slightly poor, probably because its simple linear constraint makes the agent’s understanding of task assignment a little slow. It can be seen that HSTCN has achieved the most advanced results. Based on the above analysis, HSTCN has a stable model architecture, which is also suitable for micro-dynamic environmental challenges. HSTCN outperforms other algorithms using hierarchical structure (RODE [17], ROMA [29]).

To test the function of each network of HSTCN, we tried “no-V” for HSTCN without the evaluation network (No internal rewards). “No-ST” stands for HSTCN, which does not contain the high-level Spatio-Temporal module (No observation analysis). The k value is set to the same for all three methods. It can be seen from Fig. 10(b) that the convergence speed of HSTCN without the evaluation network decreases. In other words, the supply of internal reward values makes the algorithm find the optimal policy more quickly. What’s more, the existence of the Spatio-Temporal module can reduce the oscillation of learning.

In Fig. 11(a), we compare the performance of HSTCN with other baseline algorithms on the “27m_vs_30 m” map, with the solid lines representing the median win rates. Our proposed method outperforms several other algorithms, particularly in more complex scenarios like “MMM2” and “27m_vs_30 m”, where the agent’s action space is more challenging. QTRAN

TABLE VIII
THE AVERAGE REWARDS IN STARCRAFT II OF MAP 27M_VS_30 M AND MMM2

Controller	Rewards with 27m_vs_30m	Controller	Rewards with MMM2
QTRAN [30]	5.08±0.16	LIIR [30]	7.67±0.06
QMIX [26]	3.27±0.11	QMIX [26]	3.43±0.08
ROMA [29]	12.16±0.04	ROMA [29]	5.94±0.13
RODE [17]	10.44±0.05	RODE [17]	12.67±0.09
G2ANet [24]	6.35±0.07	G2ANet [24]	13.77±0.08
GraphMix [28]	4.95±0.09	DOP [28]	14.41±0.05
HSTCN	17.05±0.04	HSTCN	18.32±0.02

and QMIX, which have flat structures, require more iterations to explore the entire decision space, whereas HSTCN's hierarchical grouping structure facilitates policy coordination across different levels, dividing the entire decision space into smaller spaces. Moreover, communication and decomposition methods like G2ANet and GraphMix, combined with GNNs, outperform QTRAN and QMIX, which only handle value decomposition and merger. However, they do not perform as well as role-based ROMA and RODE.

In Fig. 11(b), we observe the experimental results of the “MMM2” map, which is a more intricate task with multiple roles. It is worth noting that in a heterogeneous environment, the agent execution policy parameters of HSTCN are not shared. Here, HSTCN shows outstanding performance, proving to be well-suited for this complex scenario. Meanwhile, role-based algorithms such as ROMA and RODE, which employ the concept of roles, and G2ANet, which emphasizes topological relationship modeling, exhibit more flexibility in this heterogeneous cooperative task. Upon analysis, we found that ROMA performed worse than RODE because ROMA's grouping concept is dependent on mutual information, which is better suited for homogeneous agent environments. Furthermore, we introduced the LIIR based on reward value to this environment and observed that network sharing mechanisms are less effective in heterogeneous environments.

Meanwhile, it is worth noting that the “No-ST” variant exhibited commendable performance in both maps, albeit with a slightly slower learning rate compared to the complete HSTCN approach. This highlights the advantageous adaptability of the hierarchical structure in tackling complex problems. Additionally, the inclusion of the Spatio-Temporal module showcased its effectiveness in analyzing intricate information, thereby enhancing the learning efficiency of multiple agents. The results underscore the significance of the hierarchical framework and the Spatio-Temporal module in addressing complex scenarios and optimizing the overall learning process.

Therefore, based on the above two complete experiments, HSTCN has proved suitable for large-scale natural and abstract micro-simulation environments.

VI. CONCLUSION

We designed a MARL framework with hierarchical control, with two policies at different time granularities. The hierarchical framework alleviates the reward sparsity problem, supplemented

by the temporal block for information preservation. The high-level policy uses the inherent class diagram structure of the multi-agent system to learn the optimal cooperation policy and uses the Spatio-Temporal dependence model to compensate for the information loss caused by sparse connectivity and partial observability, which are common in the real-world multi-agent learning environment. Low-level policies can be applied to any value-decomposition variable, keeping the scheme of offline policy execution to apply to real-world task requirements. The experimental results show that HSTCN has good adaptability to long-trajectory training tasks. Also, as the number of agents increases, this approach is more scalable than the baseline approach and performs well in micromanaged game environments. In future research, we plan to focus on large, complex, and diverse environments, such as designing roles to adapt to multi-task environments. It is also valuable to expand on whether trained low-level policies can be transferred to other tasks. We can further optimize the algorithm and apply HSTCN to practical applications.

REFERENCES

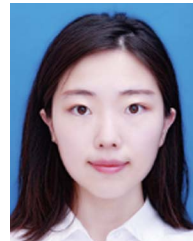
- [1] Z. Shou, X. Chen, Y. Fu, and X. Di, “Multi-agent reinforcement learning for Markov routing games: A new modeling paradigm for dynamic traffic assignment,” *Transp. Res. Part C: Emerg. Technol.*, vol. 137, 2022, Art. no. 103560.
- [2] J. Duan, S. Yu, H. L. Tan, H. Zhu, and C. Tan, “A survey of embodied AI: From simulators to research tasks,” *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 2, pp. 230–244, Apr. 2022.
- [3] J. Jin et al., “An agent-based traffic recommendation system: Revisiting and revising urban traffic management strategies,” *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 11, pp. 7289–7301, Nov. 2022.
- [4] M. Eppe, C. Gumbsch, M. Kerzel, P. D. Nguyen, M. V. Butz, and S. Wermter, “Intelligent problem-solving as integrated hierarchical reinforcement learning,” *Nature Mach. Intell.*, vol. 4, no. 1, pp. 11–20, 2022.
- [5] Y. Sun, Z. Ji, Y. Liu, and C. Lin, “On stabilizability of multi-agent systems,” *Automatica*, vol. 144, 2022, Art. no. 110491.
- [6] Y. Xie, Y. Liang, M. Gong, A. Qin, Y.-S. Ong, and T. He, “Semisupervised graph neural networks for graph classification,” *IEEE Trans. Cybern.*, early access, Apr. 27, 2022, doi: [10.1109/TCYB.2022.3164696](https://doi.org/10.1109/TCYB.2022.3164696).
- [7] N. Huang, D. Liu, Z. Sun, Z. Duan, Q. Lu, and Z. Chen, “Distributed consensus seeking with different convergence performance requirements: A unified control framework,” *IEEE Trans. Cybern.*, vol. 53, no. 9, pp. 5483–5496, Sep. 2023.
- [8] M. Lauri, D. Hsu, and J. Pajarinen, “Partially observable Markov decision processes in robotics: A survey,” *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 21–40, Feb. 2023.
- [9] W. Mao, L. Yang, K. Zhang, and T. Basar, “On improving model-free algorithms for decentralized multi-agent reinforcement learning,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 15007–15049.
- [10] P. A. Lopez et al., “Microscopic traffic simulation using sumo,” in *Proc. IEEE 21st Int. Conf. Intell. Transp. Syst.*, 2018, pp. 2575–2582.

- [11] M. Samvelyan et al., "The starcraft multi-agent challenge," in *Proc. 18th Int. Conf. Auton. Agents MultiAgent Syst.*, 2019, pp. 2186–2188.
- [12] Y. Chen, Z. Zheng, and X. Gong, "MARNet: Backdoor attacks against cooperative multi-agent reinforcement learning," *IEEE Trans. Dependable Secure Comput.*, no. 1, pp. 1–11, 2023.
- [13] O. Nachum, S. S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 3303–3313.
- [14] S. Li, R. Wang, M. Tang, and C. Zhang, "Hierarchical reinforcement learning with advantage-based auxiliary rewards," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1409–1419.
- [15] S. Ahilan and P. Dayan, "Feudal multi-agent hierarchies for cooperative reinforcement learning," in *Proc. Conf. Reinforcement Decis.*, 2019, p. 57.
- [16] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "MAVEN: Multi-agent variational exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 7613–7624.
- [17] T. Wang, T. Gupta, B. Peng, A. Mahajan, S. Whiteson, and C. Zhang, "Rode: Learning roles to decompose multi-agent tasks," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 26–30.
- [18] J. Foerster, I. A. Assael, N. D. Freitas, and S. Whiteson, "Learning to communicate with deep multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2145–2153.
- [19] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2974–2982.
- [20] W. Kim, J. Park, and Y. Sung, "Communication in multi-agent reinforcement learning: Intention sharing," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–15.
- [21] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.
- [22] C. Banerjee, Z. Chen, N. Noman, and M. Zamani, "Optimal actor-critic policy with optimized training datasets," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 6, pp. 1324–1334, Dec. 2022.
- [23] S. Sukhbaatar et al., "Learning multiagent communication with backpropagation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2252–2260.
- [24] Y. Liu, W. Wang, Y. Hu, J. Hao, X. Chen, and Y. Gao, "Multi-agent game abstraction via graph attention neural network," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 7211–7218.
- [25] P. Sunehag et al., "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proc. 17th Int. Conf. Auton. Agents MultiAgent Syst.*, 2018, pp. 2085–2087.
- [26] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4295–4304.
- [27] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5887–5896.
- [28] N. Naderizadeh, F. H. Hung, S. Soleyman, and D. Khosla, "Graph convolutional value decomposition in multi-agent reinforcement learning," 2020, *arXiv:2010.04740*.
- [29] T. Wang, H. Dong, V. Lesser, and C. Zhang, "ROMA: Multi-agent reinforcement learning with emergent roles," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9876–9886.
- [30] Y. Wang, B. Han, T. Wang, H. Dong, and C. Zhang, "DOP: Off-policy multi-agent decomposed policy gradients," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–20.
- [31] Y. Du, L. Han, M. Fang, J. Liu, T. Dai, and D. Tao, "LIIR: Learning individual intrinsic reward in multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 4403–4414.
- [32] S. Guicheng and W. Yang, "Review on dec-POMDP model for marl algorithms," in *Proc. Smart Commun. Intell. Algorithms Interactive Methods*, 2022, pp. 29–35.
- [33] W. Huang et al., "Multiagent Q-learning with sub-team coordination," in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst.*, 2022, pp. 1630–1632.
- [34] M. Hutsebaut-Buysse, K. Mets, and S. Latré, "Hierarchical reinforcement learning: A survey and open research challenges," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 1, pp. 172–221, 2022.
- [35] S. Ouaret, "Production control problem with semi-Markov jump under stochastic demands and deteriorating inventories," *Appl. Math. Modelling*, vol. 107, pp. 85–102, 2022.
- [36] B. Lyu et al., "Efficient spectral graph convolutional network deployment on memristive crossbars," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 2, pp. 415–425, Apr. 2023.

- [37] A. M. Alvi, S. Siuly, and H. Wang, "A long short-term memory based framework for early detection of mild cognitive impairment from EEG signals," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 2, pp. 375–388, Apr. 2023.
- [38] T. Zhang, Z. Liu, Z. Pu, and J. Yi, "Automatic curriculum learning for large-scale cooperative multiagent systems," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 3, pp. 912–930, Jun. 2023.
- [39] Y. Tang, L. Zhang, Q. Teng, F. Min, and A. Song, "Triple cross-domain attention on human activity recognition using wearable sensors," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 6, no. 5, pp. 1167–1176, Oct. 2022.



Tinghuai Ma received the bachelor's degree from the Huazhong University of Science and Technology (HUST), China, 1997, the master's degree from HUST, China, in 2000, and the Ph.D. degree from the Chinese Academy of Science, Beijing, China, in 2003. He was Postdoctoral Associate with AJOU University, Suwon, China, in 2004. He is currently a Professor with the School of Software, Nanjing University of Information Science & Technology, Nanjing, China. His research interests include data mining, social network, privacy preserving, data sharing.



Kexing Peng is currently working toward the Ph.D. degree with the School of Computer Science, Nanjing University of Information Science & Technology, Nanjing, China. Her research interests include deep learning and the application of multi-agent reinforcement learning, especially on the game environment and virtual reality platform.



Huan Rong received the Ph.D. degree in computer science from the Nanjing University of Information Science & Technology, Nanjing, China. He is a Visiting Scholar with the University of Central Arkansas, Conway, AR, USA. He is currently a Lecturer with the School of Artificial Intelligence, Nanjing University of Information Science & Technology. His research interests include deep learning and the application of artificial intelligence, especially on the sentiment analysis and other interdisciplinary task. His research contributions have been published in the Information Sciences, the IEEE Transactions on Affective Computing, the Soft Computing.



Yurong Qian received the B.S. and M.S. degrees in computer science and technology from Xinjiang University, China in 2002 and 2005, respectively, and the Ph.D. degree in biology from Nanjing University, Nanjing, China, in 2010. She is currently a Professor with the School of Software, Xinjiang University. From 2012 to 2013, she was a Postdoctoral Fellow with the Department of Electronics and Computer Engineering, Hanyang University, Seoul, South Korea. Her research interests include cloud computing, image processing, as well as intelligent computation

such as artificial neural networks.

Najla Al-Nabhan received the B.S. degree in computer applications (Hon) and the M.S. degree in computer science from The George Washington University, Washington, DC, USA, in 2005 and 2008 respectively, Ph.D. degree in computer science from King Saud University, Riyadh, Saudi Arabia, in 2013. She is currently the Vice Assistant Professor with the Computer Science Department, College of Computer and Information Sciences, King Saud University. Her current research interest includes: Wireless sensor networks, multimedia sensor networks, cognitive networks, and network security.